

پروژه طراحی کامپایلر

نام دانشجو: امیر مهدی محرابیان

درس: اصول طراحی کامپایلر

موضوع: اشاره گر و کاربرد آن در C++

توضیحات: میخواهیم با یک برنامه ابتدا یک مقدار عدد را وارد یک متغیر کنیم سپس با استفاده از یک متغیر اشاره گر مقدار متغیر اول را عوض کنیم و پس از آن به کمک اشاره گر مقدار و آدرس متغیر اول را بدست بیاوریم

کد:

```
int main (){  
    int mynumber = 9;  
    int *ptr= &mynumber;  
    cout<<"Variable's address with the help of pointer " <<ptr<<endl;  
    cout<<"Variable value with the help of pointer" <<*ptr<<endl;  
    *ptr = 7;  
    cout<<"New value of variable with the help of pointer: "  
    <<mynumber<<endl;  
}
```

SYMBOL TABLE

ROW	LEXEME	TYPE	VALUE	DESCRIPTION
1	int	KW	int	نوع داده عدد صحیح
2	mynumber	ID	mynumber	
3	=	S	=	نماد مقدار دهی
4	9	NUM	9	
5	;	S	;	نماد پایان یک عبارت
6	int	KW	int	نوع داده عدد صحیح
7	*	OP	*	عملگر دسترسی به متغیری که اشاره گر به آن اشاره میکند
8	ptr	ID	ptr	
9	=	S	=	نماد مقدار دهی
10	&	S	&	عملگر آدرس
11	mynumber	ID	mynumber	
12	;	S	;	نماد پایان یک عبارت
13	cout	KW	cout	برای نمایش در خروجی
14	<<	OP	<<	عملگر درج در خروجی
15	"	S	"	نماد برای نشان دادن رشته

16	Variable's address with the help of pointer:	STR	Variable's address with the help of pointer:	
17	"	S	"	نماد برای نشان دادن رشته
18	<<	OP	<<	عملگر درج در خروجی
19	ptr	ID	ptr	
20	<<	OP	<<	عملگر درج در خروجی
21	endl	KW	endl	برای رفتن به خط بعد
22	;	S	;	نماد پایان یک عبارت
23	cout	KW	cout	برای نمایش در خروجی
24	<<	OP	<<	عملگر درج در خروجی
25	"	S	"	نماد برای نشان دادن رشته
26	Variable value with the help of pointer:	STR	Variable value with the help of pointer:	
27	"	S	"	نماد برای نشان دادن رشته
28	<<	OP	<<	عملگر درج در خروجی
29	*	OP	*	عملگر دسترسی به متغیری که اشاره گر به آن اشاره میکند
30	ptr	ID	ptr	

31	<<	OP	<<	عملگر درج در خروجی
32	endl	KW	endl	برای رفتن به خط بعد
33	;	S	;	نماد پایان یک عبارت
34	*	OP	*	عملگر دسترسی به متغیری که اشاره گر به آن اشاره میکند
35	ptr	ID	ptr	
36	=	S	=	نماد مقدار دهی
37	7	NUM	7	
38	cout	KW	cout	برای نمایش در خروجی
39	<<	OP	<<	عملگر درج در خروجی
40	"	S	"	نماد برای نشان دادن رشته
41	New value of variable with the help of pointer:	STR	New value of variable with the help of pointer:	
42	<<	OP	<<	عملگر درج در خروجی
43	mynumber	ID	mynumber	
44	<<	OP	<<	عملگر درج در خروجی
45	endl	KW	endl	برای رفتن به خط بعد
46	;	S	;	نماد پایان یک عبارت

TOKEN

<int, >

<ID,2>

<=, >

<9,>

<;, >

<int, >

<*, >

<ID,8>

<=, >

<&, >

<ID,11>

<; , >

<cout, >

<<<, >

<" , >

< Variable's address with the help of pointer: , >

<" , >

<<< , >

<ID,19>

<<<, >

< endl, >

<;, >

<cout, >

<<<, >

<" , >

< Variable value with the help of pointer, >

<" , >

<<<, >

<*, >

<ID,30>

<<<, >

<endl , >

<;, >

<*, >

<ID,35>

<=, >

<cout, >

<<<, >

<' , >

< New value of variable with the help of pointer:, >

<<<, >

<ID,43>

<<< , >

<endl, >

<; , >

عبارات با قاعده کد پروژه

1-\s*

```
int\s+([a-zA-Z][a-zA-Z0-9_]*)\s*=\s*[0-9]+;\s*
```

```
# int mynumber = 9;
```

2-\s*

```
int\s+*\s+([a-zA-Z][a-zA-Z0-9_]*)\s*=\s*&\s*([a-zA-Z][a-zA-Z0-9_]*);\s*
```

```
# int *ptr = &mynumber;
```

3-\s*

```
cout\s*<<\s*"([^\"]*)" \s*<<\s*([a-zA-Z][a-zA-Z0-9_]*);\s*endl;\s*
```

```
# cout << " Variable's address with the help of pointer : " << ptr << endl;
```

4-\s*

```
cout\s*<<\s*"([^\"]*)" \s*<<\s* \s*([a-zA-Z][a-zA-Z0-9_]*);\s*endl;\s*
```

```
# cout << " Variable value with the help of pointer : " << *ptr << endl;
```

5-\s*

```
\s*\s*([a-zA-Z][a-zA-Z0-9_]*)\s*=\s*[0-9]+;\s*
```

```
# *ptr = 7;
```

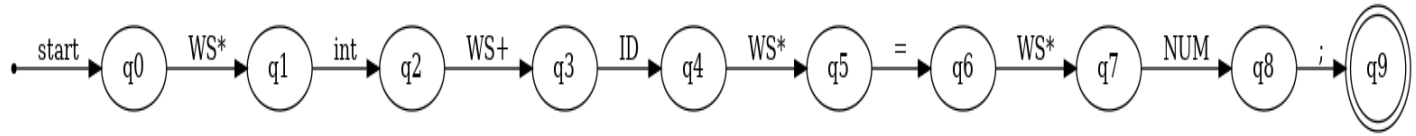
6-\s*

```
cout\s*<<\s*"([^\"]*)" \s*<<\s*([a-zA-Z][a-zA-Z0-9_]*);\s*endl;\s*
```

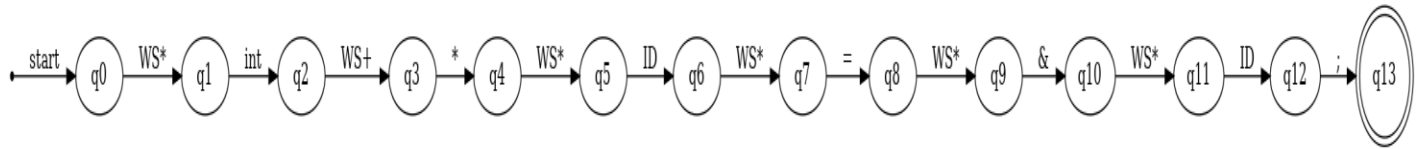
```
# cout << " New value of variable with the help of pointer : " << mynumber << endl;
```

```
\s*
```

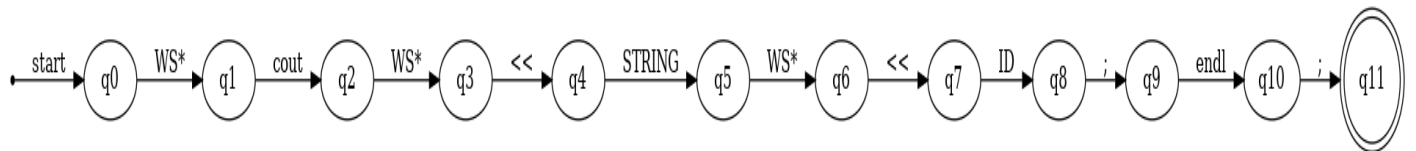
DFA part1



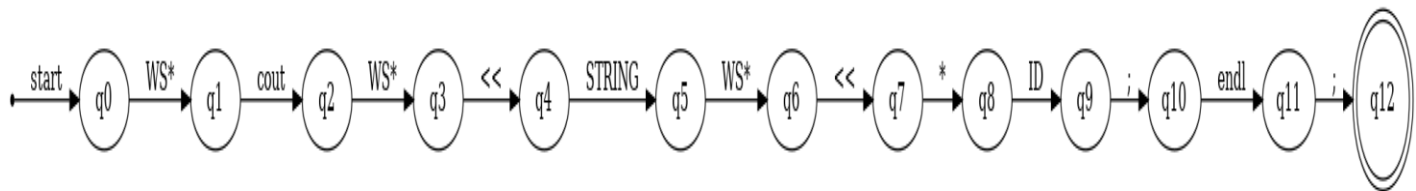
DFA part2



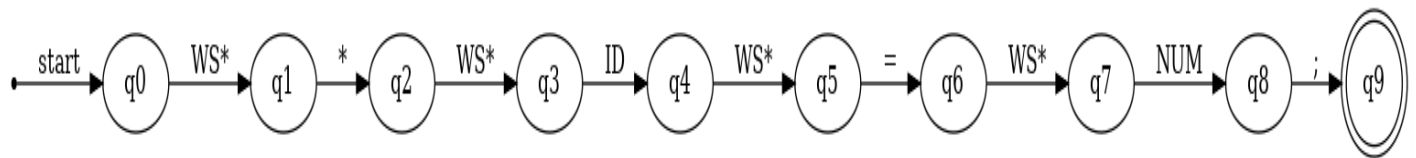
DFA part3



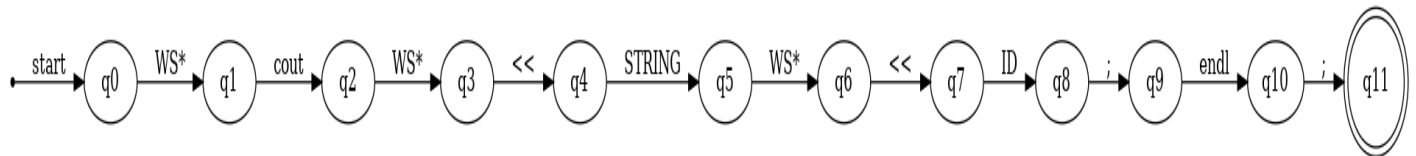
DFA part4



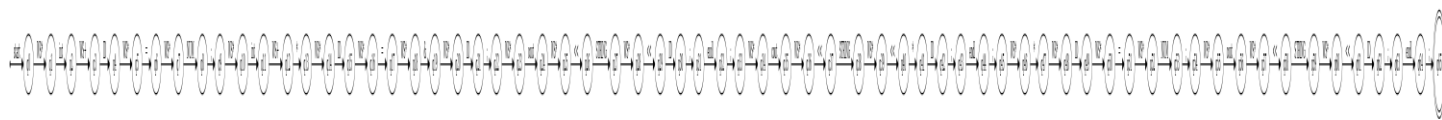
DFA part5



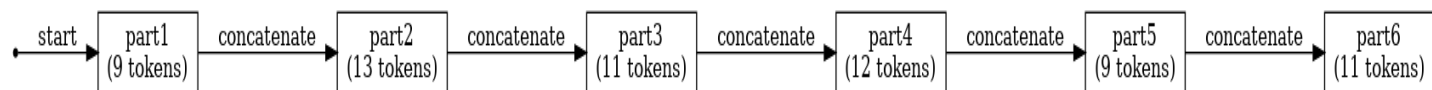
DFA part6



Combined DFA (concatenation)[zoom required]



Connected DFAs (simple)



جدول راهنمای dfa ها:

توضیح	تعریف با عبارت با قاعده	توکن
صفر یا چند فاصله و تب یا خط جدید	<code>[\t \n \r]*</code>	<code>WS*</code>
یک یا چند فاصله و تب یا خط جدید	<code>[\t \n \r]+</code>	<code>WS+</code>
شناسه (مثل متغیرها)	<code>[A-Za-z_][A-Za-z0-9_]*</code>	<code>ID</code>
عدد صحیح شامل یک یا چند رقم	<code>[0-9]+</code>	<code>NUM</code>
رشته داخل کوتیشن دوتایی	<code>"[^"]*"</code>	<code>STRING</code>
کلمه کلیدی <code>int</code> در <code>c++</code>	<code>int</code>	<code>int</code>
شیء خروجی استاندارد <code>c++</code>	<code>cout</code>	<code>cout</code>
دستور پایان خط	<code>endl</code>	<code>endl</code>
عملگر اشاره گر	<code>*</code>	<code>*</code>
عملگر ارجاع	<code>&</code>	<code>&</code>
عملگر انتساب	<code>=</code>	<code>=</code>
عملگر خروجی	<code><<</code>	<code><<</code>
پایان دستور	<code>;</code>	<code>;</code>

1- گرامر به روش EBNF

Program = { Statement } ;

Statement = Decl
 | PtrDecl
 | PrintID
 | PrintPtr
 | Assign
 | PrintFinal ;

Decl = "int" , ID , "=" , NUM , ";" ;

PtrDecl = "int" , "*" , ID , "=" , "&" , ID , ";" ;

PrintID = "cout" , "<<" , STRING , "<<" , ID , "<<" , "endl" , ";" ;

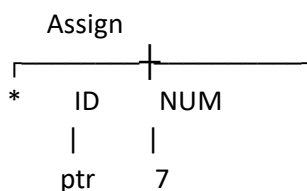
PrintPtr = "cout" , "<<" , STRING , "<<" , "*" , ID , "<<" , "endl" , ";" ;

Assign = "*" , ID , "=" , NUM , ";" ;

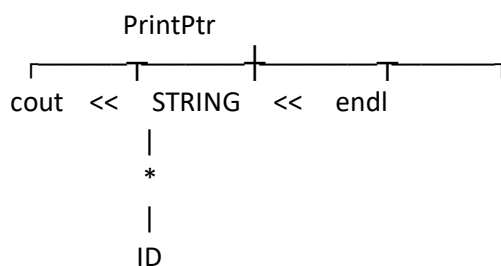
PrintFinal = "cout" , "<<" , STRING , "<<" , ID , "<<" , "endl" , ";" ;

2- درخت تجزیه (Parse Tree)

درخت تجزیه برای دستور: *ptr = 7;



درخت تجزیه برای دستور: cout << "text" << *ptr << endl;



|
ptr

FOLLOW3- FIRST

$\text{FIRST}(\text{Statement}) = \{ \text{int}, \text{cout}, * \}$

$\text{FOLLOW}(\text{Statement}) = \{ \text{int}, \text{cout}, *, \$ \}$

4- جدول تجزیه (LL(1) Parsing Table)

Non-Terminal	int	cout	*	\$
Statement	Decl / PtrDecl	PrintID / PrintPtr / PrintFinal	Assign	Error

5- نتیجه گیری

گرامر EBNF استخراج شده از عبارات با قاعده ارائه شده، قادر است ساختارهای اصلی برنامه شامل اعلان متغیر، اعلان اشاره گر، انتساب از طریق اشاره گر و دستورات خروجی را تجزیه نماید.